Documentacao Tecnica: Evolucao do Sistema CittaLixo

Escopo: Identificacao de Ecopontos e Horarios de Coleta Domiciliar em Olinda e Recife (100% Gratuito)

Repositorio: https://github.com/renansousones020-sudo/cittalixo

1. Problemas Diagnosticados no Codigo Original e Correcoes Aplicadas

- **Desconexao entre Front-end e Back-end:** Antes, o index.html exibia apenas 2 pontos fixos no HTML e nao consumia a API FastAPI. Agora o front-end faz requisicoes dinamicas (fetch('/pontos')) e reflete criacoes, filtros e atualizacoes em tempo real.
- **Coordenadas Restritas ao Centro do Recife:** O sistema original apontava apenas para uma coordenada no Recife. Foi expandido para cobrir Olinda (Bairro Novo, Casa Caiada, Peixinhos, Carmo/Sitio Historico, Ouro Preto, Rio Doce, Varadouro) e Recife (Boa Vista, Campo Grande, Torre, Imbiribeira, Casa Amarela).
- **Colisao de IDs na Criacao de Pontos:** O calculo anterior (len(pontos) + 1) gerava IDs duplicados apos qualquer exclusao. Foi substituido por chave primaria autoincrementada (INTEGER PRIMARY KEY AUTOINCREMENT) no SQLite.
- **Perda de Dados ao Reiniciar (Lista em Memoria):** A lista em memoria foi substituida pelo banco local SQLite (cittalixo.db), nativo do Python e sem custo, ja inicializado automaticamente com 13 pontos reais/representativos de Olinda e Recife.
- **Codigos de Status HTTP REST Incorretos:** Buscas, atualizacoes e remocoes de IDs inexistentes antes retornavam 200 OK. Agora retornam 404 Not Found via HTTPException, e o POST /pontos retorna 201 Created.
- **Codificacao UTF-16LE no requirements.txt:** O arquivo requirements.txt estava em UTF-16LE (quebrando builds Linux/Docker). Foi convertido para UTF-8 limpo com apenas as dependencias necessarias.

2. Novas Funcionalidades Implementadas (Arquitetura Custo Zero)

- **Ecopontos Fixos + Horarios de Coleta Domiciliar:** Suporte simultaneo a pontos fixos de descarte (marcadores verdes no mapa) e rotas/horarios do caminhao de coleta domiciliar por bairro (marcadores azuis).
- **Horarios Detalhados e Indicador 'Aberto Agora / Coleta Hoje':** Cada ponto possui horario descritivo, dias da semana (Seg a Sab/Dom) e janela de horario (hora_inicio e hora_fim). O navegador calcula automaticamente se o ponto esta aberto ou com coleta em andamento no momento do acesso.
- **Filtros por Cidade (Olinda / Recife), Modalidade, Residuo e Busca:** Botoes rapidos para alternar entre Olinda, Recife ou ambas as cidades (com ajuste automatico de zoom no mapa), alem de filtros para Reciclaveis, Eletronicos, Entulho, Oleo de Cozinha e Lixo Organico.
- **Botao 'Perto de Mim' (Geolocalizacao GPS + Haversine):** Utiliza a API gratuita navigator.geolocation do navegador e calcula a distancia em quilometros (km) ate cada ponto sem precisar de chaves pagas da Google Distance Matrix API.

- **Cadastro Colaborativo com Geocodificacao Gratuita:** Formulario integrado na pagina onde moradores podem cadastrar novos pontos clicando direto no mapa ou buscando o endereco gratuitamente via OpenStreetMap Nominatim.
- **Botao 'Como Chegar':** Link direto em cada card e popup do mapa para abrir a navegacao por rota ate o ponto de coleta.

3. Como e Onde Testar o Sistema CittaLixo

Abaixo estao todas as formas de validar e demonstrar o funcionamento do projeto:

- Opcao A - Teste Visual no Navegador (Aplicacao Web e Mapa): Com o servidor Uvicorn ativo, acesse no navegador para testar os filtros de Olinda e Recife, clicar nos marcadores do mapa, usar o botao 'Perto de Mim' e cadastrar um novo ponto:

```
# 1. Iniciar o servidor na pasta Cittalixo-main:
..\.venv\Scripts\uvicorn.exe main:app --host 127.0.0.1 --port 8000 --reload

# 2. Acessar no navegador:
Aplicacao Web (Mapa Interativo): http://127.0.0.1:8000
Documentacao Interativa (Swagger UI): http://127.0.0.1:8000/docs
Health Check (Observabilidade JSON): http://127.0.0.1:8000/health
```

- Opcao B - Testes Automatizados de API (Pytest + FastAPI TestClient): Valida automaticamente o endpoint /health, listagem geral, filtros por cidade (Olinda e Recife) e categoria, ciclo CRUD completo (POST 201, GET 200, PUT 200, DELETE 200) e tratamento de erro 404:

```
cd C:\Users\renan.andrade\Desktop\cittalixo\Cittalixo-main
..\.venv\Scripts\pytest.exe test_api.py -v
```

- Opcao C - Testes End-to-End de Interface (Selenium Headless): Abre o navegador Chrome de forma automatizada (modo headless), acessa http://127.0.0.1:8000 e verifica o titulo, a renderizacao do mapa Leaflet, o carregamento dinamico dos cards da API (Recife e Olinda) e a exibicao dos horarios:

```
# Com o servidor rodando na porta 8000, execute em outro terminal:
cd C:\Users\renan.andrade\Desktop\cittalixo\Cittalixo-main
..\.venv\Scripts\python.exe testes_cittalixo.py
```

- Opcao D - Integracao Continua na Nuvem (GitHub Actions CI): A cada git push na branch main, o workflow .github/workflows/ci.yml sobe um ambiente Ubuntu na nuvem do GitHub, instala as dependencias do requirements.txt e executa toda a suite do pytest automaticamente:

```
URL do CI: https://github.com/renansousones020-sudo/cittalixo/actions
```

- Opcao E - Execucao em Container Docker: O projeto possui um Dockerfile otimizado (Python 3.11-slim) pronto para execucao local ou deploy gratuito em plataformas como Render ou Railway:

```
docker build -t cittalixo .
docker run -p 8000:8000 cittalixo
```